

Relatório de Laboratório

Laboratório de Experimentação de Software

Curso	Engenharia de Software
Disciplina	Laboratório de Experimentação de Software
Turno / Período	Noite / 6º
Professor(a)	Danilo Maia
Laboratório	Lab01 — Características de repositórios populares + Setup do Kanban
Grupo (trio)	Athos Marques Ribeiro Fonseca · Mateus Araujo Santos · Rafael Ganascini de Moura
Link do repositório / GitHub Projects	Repositório: https://github.com/RafaelMouraG/Laboratorio-de-Experimentacao-de-Software GitHub Projects: https://github.com/users/RafaelMouraG/projects/6
Data de entrega	27/08/2026

1. Introdução

Este laboratório inicia a disciplina com dois objetivos que se sustentam um ao outro: caracterizar empiricamente os repositórios open-source mais populares do GitHub e implantar o processo de trabalho (GitHub Projects, política de WIP, snapshots de sprint) que o grupo vai reutilizar nos laboratórios seguintes. A motivação de fundo é a mesma que orienta a disciplina: decisões sobre "o que faz um projeto de software bem-sucedido" costumam se apoiar em intuição — idade, atividade, adoção de determinada linguagem — sem evidência controlada por trás. Este laboratório testa sete dessas intuições contra os 1.000 repositórios com mais estrelas no GitHub.

Questões de Pesquisa do enunciado:

- RQ01. Sistemas populares são maduros/antigos? Métrica: idade do repositório (createdAt).
- RQ02. Sistemas populares recebem muita contribuição externa? Métrica: total de pull requests aceitas (merged).
- RQ03. Sistemas populares lançam releases com frequência? Métrica: total de releases.
- RQ04. Sistemas populares são atualizados com frequência? Métrica: tempo até a última atualização (pushedAt).
- RQ05. Sistemas populares são escritos nas linguagens mais populares? Métrica: linguagem primária de cada repositório, comparada ao top 10 do TIOBE Index.
- RQ06. Sistemas populares possuem um alto percentual de issues fechadas? Métrica: razão entre issues fechadas e total de issues.
- RQ07. Sistemas escritos em linguagens mais populares recebem mais contribuição externa, lançam mais releases e são atualizados com mais frequência? Cruzamento das RQ02/RQ03/RQ04 por linguagem.

Hipóteses informais do grupo, formuladas antes da análise dos 1.000 repositórios:

- RQ01 — repositórios populares devem ser majoritariamente maduros, mas com uma cauda de projetos recentes que viralizaram rápido.
- RQ02 — espera-se volume alto de PRs aceitas na mediana, com o viés conhecido de projetos que não usam PR do GitHub como fluxo principal de contribuição aparecerem com zero.
- RQ03 — espera-se frequência alta de releases, mas de forma bimodal: uma parte relevante da amostra não usa GitHub Releases (por não ser software ou por versionar por tag); entre os que publicam, a cadência deve ser alta.
- RQ04 — espera-se atualização frequente na mediana, convivendo com uma cauda de projetos arquivados que seguem estrelados por reputação acumulada.
- RQ05 — espera-se que os projetos de maior sucesso sejam escritos predominantemente nas linguagens que dominam o mercado, referência TIOBE Index, pelo tamanho do ecossistema de bibliotecas e de desenvolvedores aptos a contribuir.
- RQ06 — espera-se taxa de issues fechadas muito alta (mediana acima de 80%), como reflexo de manutenção ativa.
- RQ07 — a leitura natural é que sim, pelo mesmo argumento estrutural da RQ05: linguagem popular teria o maior ecossistema e a maior massa de contribuidores, o que deveria puxar as três métricas para cima.

Além do enunciado, o grupo propõe uma questão adicional (detalhada na seção 3.6 — Inovações): RQ extra — a idade do repositório explica seu número de estrelas? A dúvida surgiu ao observar, já na validação da RQ01, que a faixa etária mais jovem concentrava a maior mediana de estrelas da amostra.

2. Contexto

Este é o Lab01, o primeiro laboratório do semestre: além da mineração em si, ele estabelece a infraestrutura de processo (GitHub Projects do grupo, política de WIP e o script de snapshot de sprint) que será reutilizada, e cuja série histórica de snapshots alimentará diretamente os Labs 04 e 05.

O objeto de estudo são os 1.000 repositórios com maior número de estrelas no GitHub, coletados via API GraphQL própria do grupo (sem bibliotecas de terceiros para consulta à API, conforme exigido pelo enunciado). A amostra evoluiu em duas etapas: 100 repositórios na sprint S01 (teto inicial, usado para validar campos e hipóteses) e 1.000 na sprint S02 — o teto de resultados que o endpoint de busca do GitHub devolve.

Referência conceitual usada como base para a RQ05 e a RQ07: o TIOBE Index (<https://www.tiobe.com/tiobe-index/>), consultado na edição de 2026, como fonte para "linguagens mais populares". A mesma referência é mantida do início ao fim do laboratório, conforme recomendado pelo enunciado.

3. Metodologia

3.1 Principais Desafios

- Truncamento silencioso do totalCount em releases. Sem orderBy explícito, a API GraphQL do GitHub trunca releases.totalCount em 1000 — vercel/next.js retornava 1000 em vez de 3.800 e ggml-org/llama.cpp, 1000 em vez de 6.855. A correção (orderBy: {field: CREATED_AT, direction: DESC}) só foi percebida porque os scripts passaram a alertar sempre que um repositório aparecesse com exatamente 1000 releases — sintoma do bug, não um valor real.
- Paginação instável na faixa dos repositórios mais pesados. Entre as posições 76-100 da amostra (kubernetes/kubernetes, electron/electron, llama.cpp), a API respondia 502/503/504 com páginas de 25 itens. A solução foi paginação adaptativa: começar com páginas de 25 e reduzir pela metade até a API aceitar, com retry e pausa quando o rate limit está perto do limite — lógica centralizada em graphql_pagination.py e reutilizada pelos dois scripts que paginam sobre mais de uma página.
- GitHub Projects não guarda histórico de coluna consultável via API. A API só devolve o status atual de cada cartão, então não é possível reconstruir retroativamente o fluxo do board. A solução adotada foi rodar um snapshot GraphQL (export_snapshot.py) ao fim de cada sprint, acumulando uma série de CSVs versionados que serve de base para os Labs 04 e 05.
- Coluna primary_language com valor ausente. Repositórios sem linguagem principal definida gravam a string "N/A", que o pandas lê como NaN por padrão — o groupby descartava essas linhas em silêncio (87 dos 1.000 repositórios). A leitura precisa ser feita com keep_default_na=False em toda a análise.

3.2 Tomadas de Decisão

- Limite de WIP em Doing = 3, um cartão em andamento por integrante do trio. Como cada RQ é desenvolvida individualmente dentro da própria Issue, o gargalo real do grupo é pessoa, não tarefa — então o limite acompanha o tamanho do trio, e não um número arbitrário maior.
- Fonte fixa para "linguagens mais populares" (RQ05/RQ07): TIOBE Index, mantida do início ao fim do laboratório para não misturar critérios diferentes de popularidade entre as RQs.
- Métrica de atualização por pushedAt, não updatedAt (RQ04). updatedAt muda com qualquer alteração do repositório (receber estrela, editar label ou descrição) e, medido nos 1.000, faria a cauda de abandonados desaparecer por completo (mediana cairia de 3,02 dias para 0,03 dia). Só pushedAt reflete atividade de desenvolvimento de fato.
- Métrica principal de releases é releases_por_ano, não o total bruto (RQ03/RQ07). O total acumulado favorece repositório antigo; dividir pela idade do repositório normaliza a comparação entre projetos de idades diferentes, o que pesa especialmente na RQ07, onde os grupos comparados têm composição etária distinta.
- Corte de --min-repos 10 na tabela por linguagem (RQ03/RQ07). Abaixo de 10 repositórios, a "mediana" de uma linguagem é o próprio repositório isolado — 11 das 44 linguagens nos 1.000 aparecem uma única vez. O corte evita ler ruído como tendência; a tabela agregada por grupo (top-10 TIOBE vs. demais), que não é filtrada, continua sendo a resposta oficial da RQ07.
- Grupo "sem linguagem primária" mantido separado de "demais linguagens" (RQ05/RQ07). São repositórios de conteúdo (listas, livros, roadmaps) e não uma linguagem impopular; somá-los às "demais" puxaria as medianas de releases e de atividade para baixo e mediria proporção de conteúdo em vez de linguagem.
- Exclusão de repositórios com total_issues == 0 do cálculo da RQ06. A coleta grava closed_issues_ratio = 0,0 nesses 43 casos, mas eles não usam o rastreador de issues do GitHub — não são projetos com 0% de resolução. Incluí-los derrubaria a mediana artificialmente.

3.3 Etapas

O board do grupo tem 3 integrantes; a divisão por RQ segue a sugestão do enunciado (uma RQ-par por pessoa) e é a mesma refletida nos Assignees das Issues do GitHub Projects.

Sprint	Entregas	Responsável(is)	Issues (nº)
Lab01S01	Consulta GraphQL para 100 repositórios (RQ01-RQ07) + script próprio de requisição + GitHub Projects criado com colunas Backlog→To Do→Doing→Review→Done e WIP=3 definidos.	Mateus (RQ01/RQ02) · Rafael (RQ03/RQ04, RQ07 bônus) · Athos (RQ05/RQ06, unificação do script)	#2, #3, #4, #5, #6, #7, #8, #9, #10, #11, #12
Lab01S02	Paginação para os 1.000 repositórios + dados em .csv + validação individual por	Mateus (paginação, RQ01/RQ02) · Rafael (RQ03/RQ04, snapshot) · Athos (RQ05/RQ06,	#13, #14, #15, #16, #17, #18, #19

	dupla de RQ + primeira versão do relatório com hipóteses informais + snapshot de fechamento da sprint.	relatório S02)	
Lab01S03	Análise e visualização de dados (gráficos e tabelas) para as 7 RQs.	Mateus (RQ01/RQ02) · Rafael (RQ03/RQ04, seção Resultados) · Athos (RQ05/RQ06, seção Discussão)	#24, #25, #26, #27, #28, #29, #30, #31, #32, #33
Relatório Final	Elaboração do documento final consolidado (este documento), com anexo do print do board mostrando o fluxo completo do Lab01 e a política de WIP em uso.	Grupo (Athos, Mateus, Rafael)	a definir

Configuração do processo

O board do grupo (GitHub Projects v2) usa o campo Status com as cinco colunas exigidas pelo enunciado: Backlog → To Do → Doing → Review → Done. Todo cartão é uma Issue de verdade do repositório, com Assignee — sem draft issues soltas, para que tudo seja rastreável pela API.

Limite de WIP em Doing: 3, um cartão em andamento por integrante do trio. Como cada RQ é desenvolvida individualmente dentro da própria Issue, o gargalo do grupo é pessoa, não tarefa; passar de 3 significaria alguém com duas frentes abertas ao mesmo tempo.

Ao final de cada sprint, o script `kanban/export_snapshot.py` (GraphQL, sem biblioteca de terceiros) exporta os itens do board e seu status para um CSV acumulado em `kanban/snapshots/` — como a API do GitHub Projects não guarda histórico de coluna, essa série de snapshots faz esse papel e alimenta os Labs 04 e 05. Snapshots já registrados: S01 (13/08/2026, 11 itens) e S02 (20/08/2026).

Print do board (GitHub Projects) ao final do Lab01, dividido em duas capturas para cobrir as cinco colunas: Backlog (0/10), To Do (1), Doing (1/3, dentro do WIP=3) e Review (4/5) na primeira imagem; Done (27) completando a segunda.

RafaelMouraG

Projects

Laboratório de Experimentação de Software

Q Type to search

Laboratório de Experimentação de Software

Add status update

Insights

Workflows 7

Backlog

Priority board

Team items

Roadmap

My items

New view

Q Filter by keyword or by field

View

Backlog0 / 10Estimate: 0

This item hasn't been started

To Do1Estimate: 0

This is ready to be picked up

Laboratorio-de-Experimentacao-de-Software #33

Snapshot de fechamento da sprint S03

+ Add item

Doing1 / 3Estimate: 0

This is actively being worked on

Laboratorio-de-Experimentacao-de-Software #31

Consolidar a seção 3 (Resultados) do relatório

Review4 / 5Estimate: 0

This item is in review

Laboratorio-de-Experimentacao-de-Software #29

RQ05: visualização e resultado das linguagens primárias

Laboratorio-de-Experimentacao-de-Software #36

Validar a RQ07 nos 1.000, gerar os dados por linguagem e escrever a hipótese informal

Laboratorio-de-Experimentacao-de-Software #30

RQ06: visualização e resultado do percentual de issues fechadas

Laboratorio-de-Experimentacao-de-Software #32

Consolidar a seção 4 (Discussão: hipótese x resultado)

 View

+

 **Done** 27

Estimate: 0

... +

+

This has been completed



Laboratorio-de-Experimentacao-de-
Software #37



RQ07: visualização e resultado das
RQ02/RQ03/RQ04 por linguagem



Laboratorio-de-Experimentacao-de-
Software #1



Subir documentação do repo



Laboratorio-de-Experimentacao-de-
Software #26

...



RQ02: visualização e resultado das pull
requests aceitas



Laboratorio-de-Experimentacao-de-
Software #24



Infraestrutura de gráficos: módulo comum e
pasta de figuras



Laboratorio-de-Experimentacao-de-
Software #25



RQ01: visualização e resultado da idade dos
repositórios



Laboratorio-de-Experimentacao-de-



3.4 Ferramentas

- API GraphQL do GitHub, consumida por script Python próprio do grupo (sem bibliotecas de terceiros para consulta à API) — `src/collection/collect_all_rqs.py`, com paginação adaptativa isolada em `src/collection/graphql_pagination.py`.
- Python 3.9 e Pandas para carga, limpeza e agregação dos dados coletados (leitura padronizada em `viz_common.carregar_dados`, sempre com `keep_default_na=False`).
- Matplotlib/Seaborn para as visualizações (`src/analysis/viz_*.py`), com estilo, paleta e função de marcação de mediana centralizados em `src/analysis/viz_common.py`.
- GitHub Projects (v2) como ferramenta de processo — <https://github.com/users/RafaelMouraG/projects/6> — com exportação de snapshot via GraphQL própria (`kanban/export_snapshot.py`).

3.5 Tabela de Métricas

RQ	Métrica	Definição Operacional	Unidade	Ferramenta / Fonte
RQ01	Idade do repositório	Data atual – createdAt	Anos	Script GraphQL (API do GitHub)
RQ02	Total de pull requests aceitas	<code>pullRequests(states: MERGED).totalCount</code>	PRs	Script GraphQL (API do GitHub)
RQ03	Total de releases	<code>releases(orderBy: CREATED_AT).totalCount</code>	Releases	Script GraphQL (API do GitHub)
RQ04	Tempo até a última atualização	Data atual – pushedAt	Dias	Script GraphQL (API do GitHub)
RQ05	Linguagem primária	<code>primaryLanguage.name</code> , comparada ao top 10 do TIOBE Index	Categórica	Script GraphQL (API do GitHub) + TIOBE Index
RQ06	Razão de issues fechadas	<code>issues(states: CLOSED).totalCount / issues.totalCount</code>	Razão (%)	Script GraphQL (API do GitHub)
RQ07	RQ02/RQ03/RQ04 agregadas por linguagem	Mediana de cada métrica, agrupada por top-10 TIOBE / demais / sem linguagem primária	Medianas por grupo	Cruzamento dos CSVs acima (<code>analyze_rq07.py</code> , sem coleta nova)

3.6 Inovações Propostas pelo Grupo (30% da nota)

- (a) Nova Questão de Pesquisa — a idade do repositório explica seu número de estrelas? Motivada pela observação, durante a validação da RQ01, de que a faixa etária mais jovem (< 1 ano) concentrava a maior mediana de estrelas da amostra. O script `extra_idade_vs_popularidade.py` mede correlação de Pearson e Spearman entre idade e estrelas e quebra a amostra por faixa etária; resultado e discussão aparecem nas seções 4 e 5.
- (b) Métrica adicional — `releases_por_ano` (`releases / idade do repositório`), não pedida pelo enunciado (que pede apenas o total de releases). Normaliza a comparação entre repositórios de idades diferentes

e é a métrica usada para responder RQ03 e para compor a RQ07, em vez do total bruto, que favorece repositório antigo.

- (c) Arquitetura de coleta — um único script consolidado (`collect_all_rqs.py`) que coleta as sete RQs de uma vez, com paginação adaptativa e retry (`graphql_pagination.py`) para lidar com timeouts 502/504 nos repositórios mais pesados. Os scripts de coleta por dupla de RQ (usados só na validação individual de 5-10 repositórios) foram descontinuados como fonte de dados final, para que toda a análise leia de uma única fonte e não haja divergência entre coletas feitas em momentos diferentes.
- (d) Segmentação adicional na RQ07 — em vez de só comparar "top-10 TIOBE vs. demais", o grupo isolou um terceiro grupo ("sem linguagem primária") e aplicou um corte de `--min-repos 10` na tabela por linguagem, para não deixar uma linguagem com um único repositório aparecer como tendência. Esse recorte foi o que permitiu identificar que a composição do grupo "demais" (TypeScript + Go) é a causa real da resposta negativa da RQ07, não a popularidade da linguagem em si.
- (e) Infraestrutura de processo reutilizável — o script de snapshot do Kanban (`kanban/export_snapshot.py`) via GraphQL, entregue já no Lab01 mas desenhado para alimentar os Labs 04 e 05, que dependem da série histórica de status dos cartões que o GitHub Projects não expõe nativamente.

4. Resultados

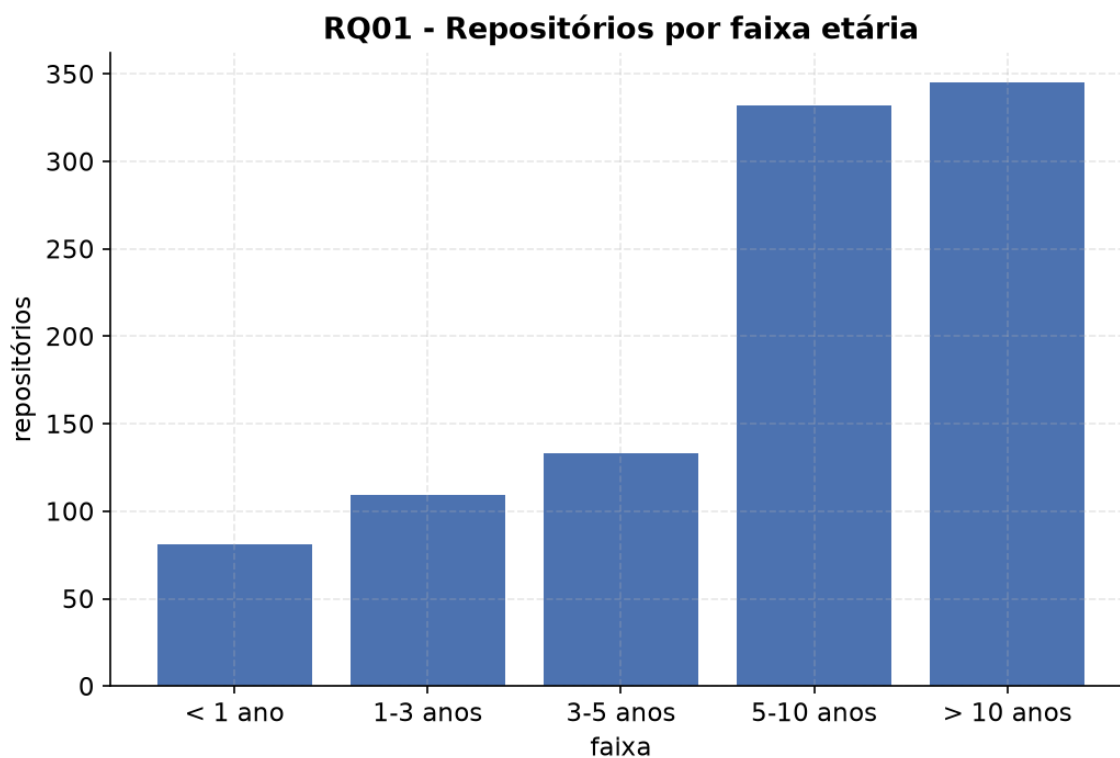
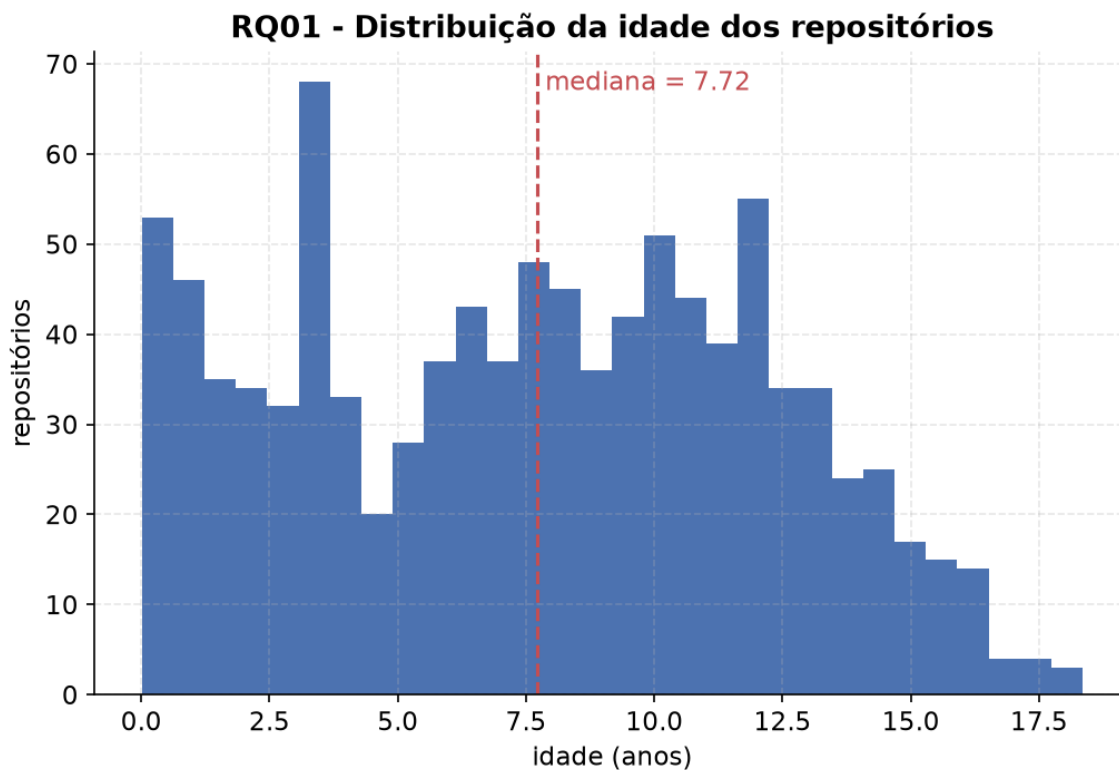
4.1 Coleta de Dados

- Volume final: 1.000 repositórios coletados na sprint S02 (teto do endpoint de busca do GitHub), todos com as sete métricas preenchidas — nenhuma célula vazia, valor não numérico ou negativo em `age_years`, `merged_pull_requests`, `releases`, `days_since_last_push` ou `days_since_last_update`. Execução de referência: 1min49s, 40 requisições (páginas de 25, sem nenhuma reduzida por 502/504).
- RQ02 — 20 dos 1.000 repositórios (2%) vêm com zero pull requests aceitas; tratado como recorte declarado (projetos que aceitam contribuição fora do fluxo de PR do GitHub, como `torvalds/linux`), não como dado ausente.
- RQ03 — 275 dos 1.000 (27,5%) não publicam nenhuma release; separados como grupo próprio para calcular a mediana com e sem os zeros, em vez de descartados.
- RQ05 — 87 dos 1.000 (8,7%) sem linguagem primária definida ("N/A"); mantidos como terceiro grupo na RQ07, em vez de somados às demais linguagens.
- RQ06 — 43 dos 1.000 repositórios não usam o rastreador de issues do GitHub (`total_issues == 0`); excluídos do cálculo da mediana de fechamento para não produzir uma taxa de 0% artificial.
- Nenhum outlier foi removido da amostra: os valores extremos identificados pelo critério de Tukey em `releases`, `releases_por_ano` e `days_since_last_push` (cauda longa em repositórios com release diária ou parados há anos) são mantidos e discutidos como parte da resposta de cada RQ, não como erro de coleta.

4.2 Visualização Gráfica

RQ01 — Sistemas populares são maduros/antigos?

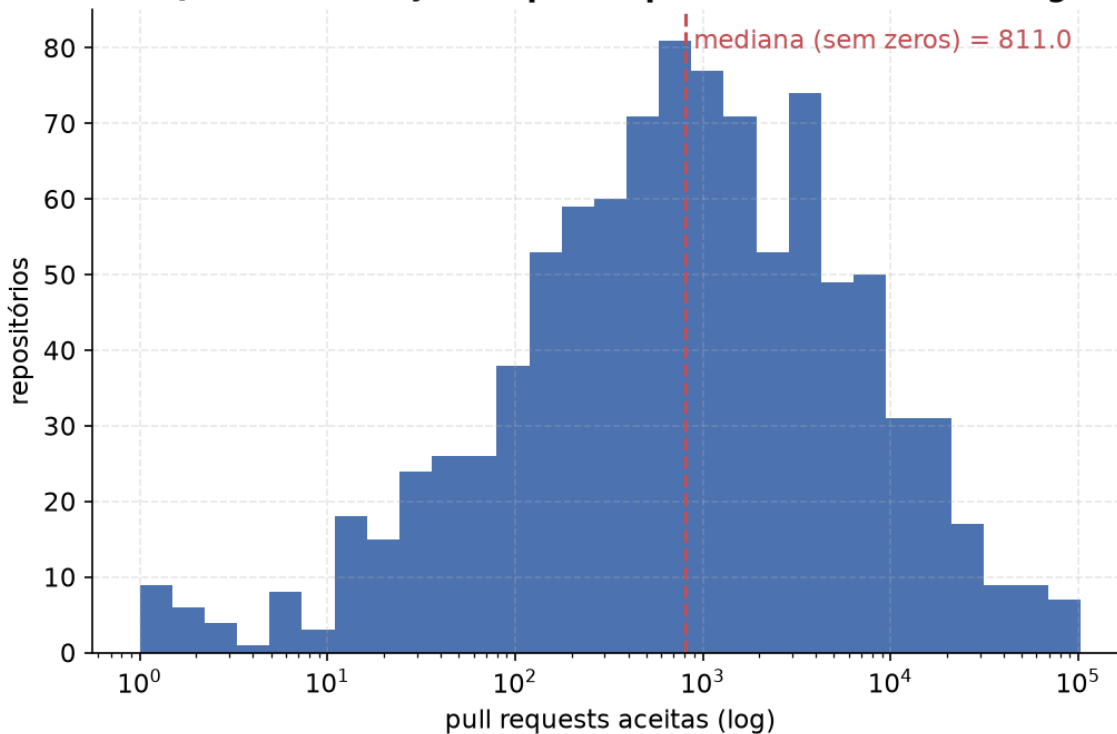
Idade mediana de 7,72 anos (Q1 3,51, Q3 11,34; mínimo 0,01, máximo 18,35). 81 dos 1.000 repositórios (8,1%) têm menos de um ano de vida, contra 345 (34,5%) com mais de dez anos — a cauda de projetos recentes prevista na hipótese é uma fração estável da amostra, não ruído.



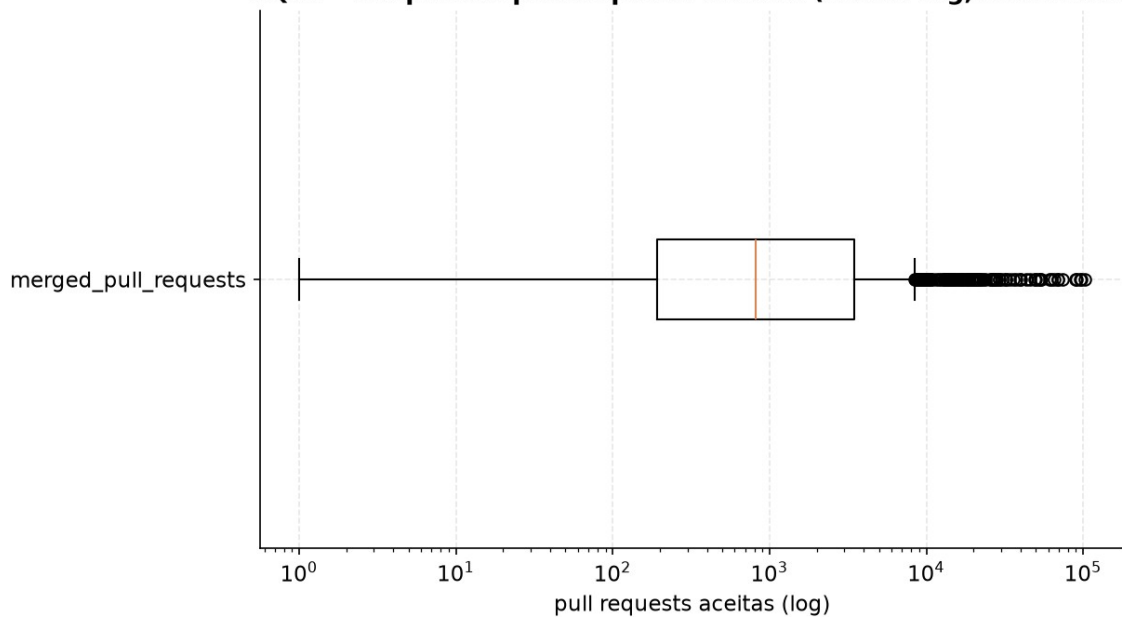
RQ02 — Sistemas populares recebem muita contribuição externa?

Mediana de 765,5 PRs aceitas considerando todos os 1.000 repositórios (Q1 175, Q3 3.390); excluindo os 20 (2%) com zero, a mediana sobe para 811. O zero não indica baixa contribuição: são projetos como torvalds/linux, que aceitam patch por lista de e-mail em vez de Pull Request.

RQ02 - Distribuição de pull requests aceitas (escala log)



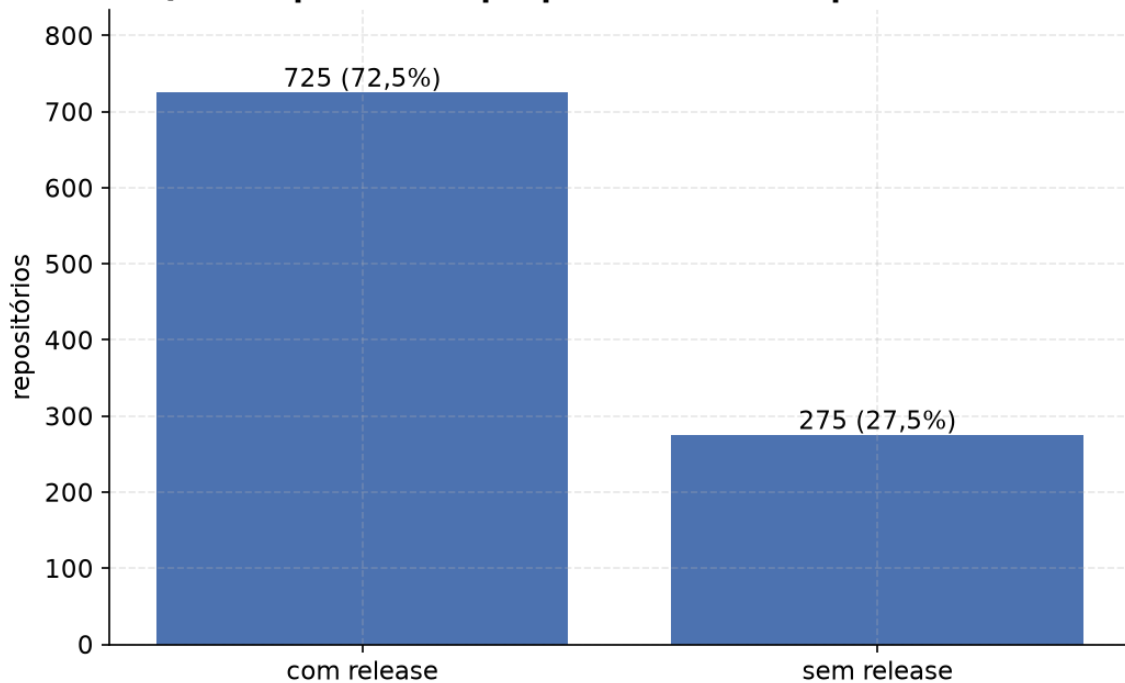
RQ02 - Boxplot de pull requests aceitas (escala log, sem zeros)



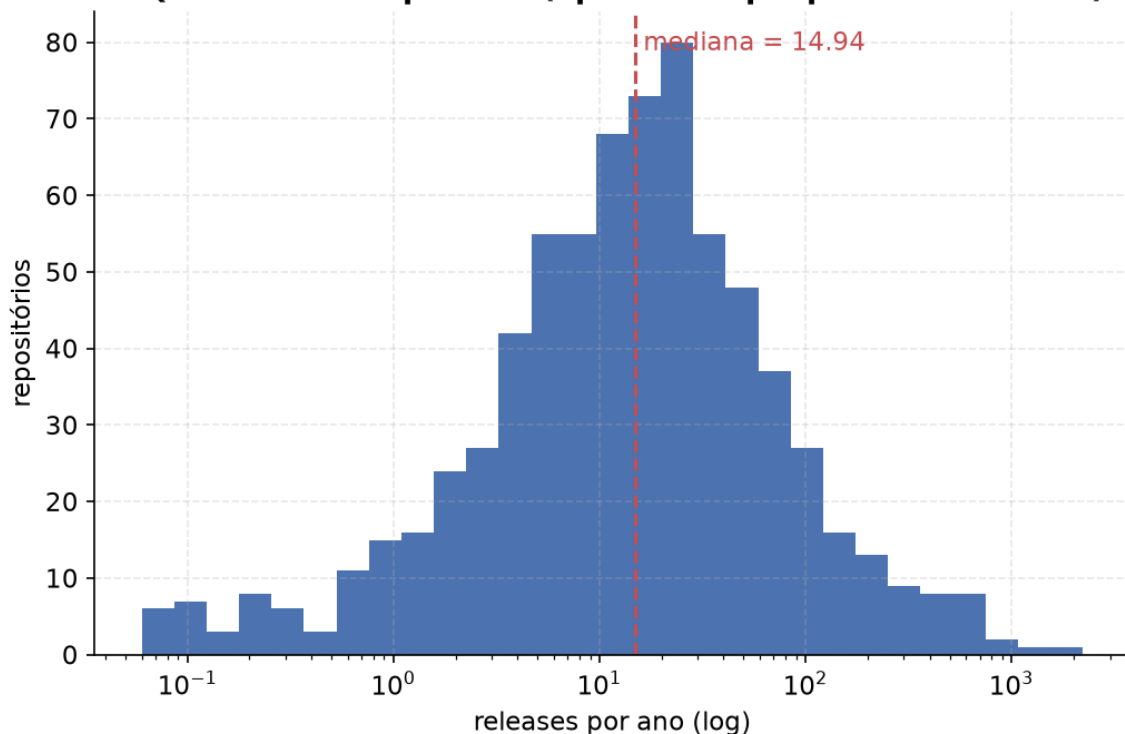
RQ03 — Sistemas populares lançam releases com frequência?

Distribuição bimodal: 275 dos 1.000 (27,5%) não publicam nenhuma release, e a mediana de releases_por_ano entre os 725 que publicam é 14,94 (6,78 considerando todos). O percentual sem release caiu de 40% (amostra de 100) para 27,5% (1.000) e varia fortemente por linguagem, de 90,9% em HTML a 5,3% em Go e Rust.

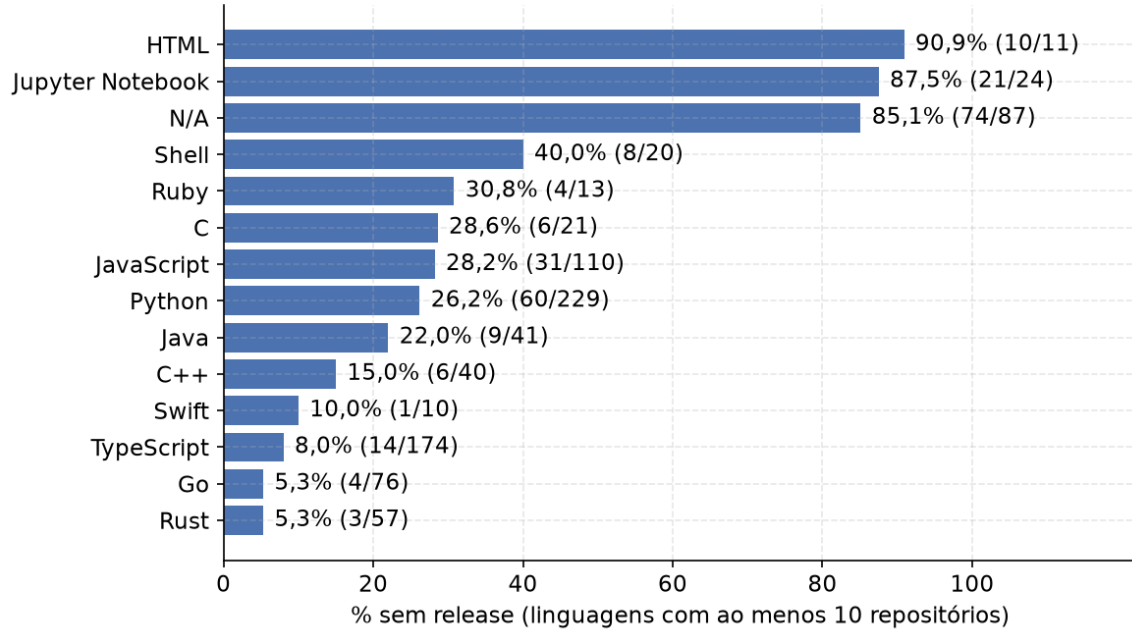
RQ03 - Repositórios que publicam vs. não publicam release



RQ03 - Releases por ano (apenas os que publicam release)



RQ03 - Repositórios sem release por linguagem primária

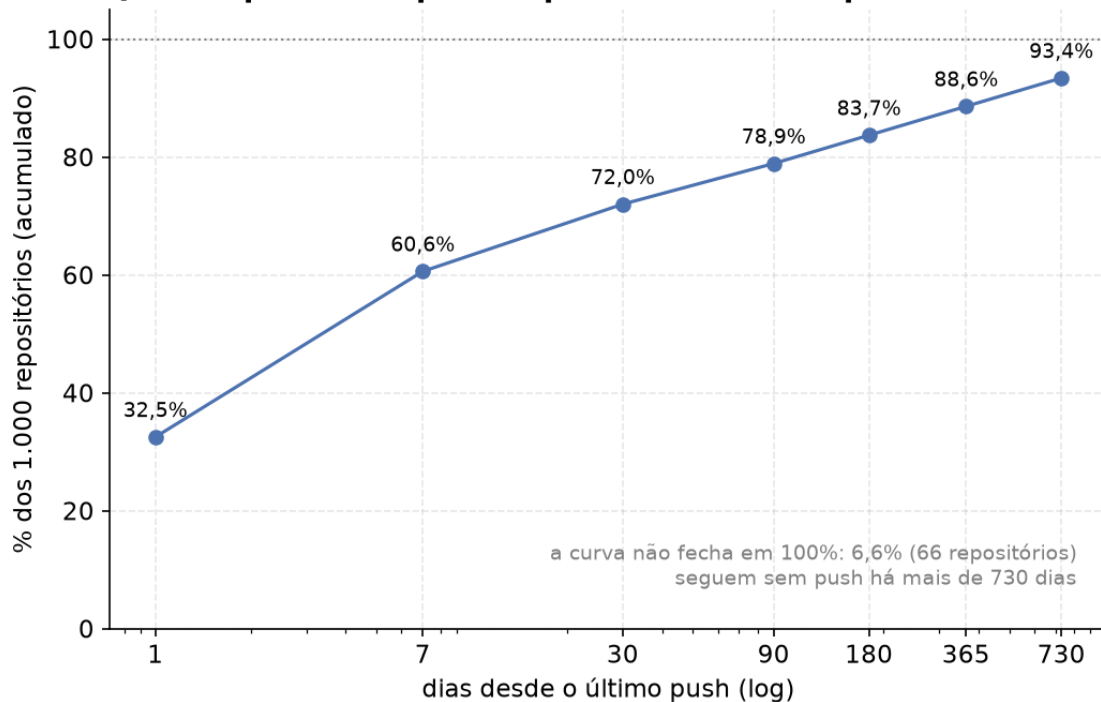


RQ04 — Sistemas populares são atualizados com frequência?

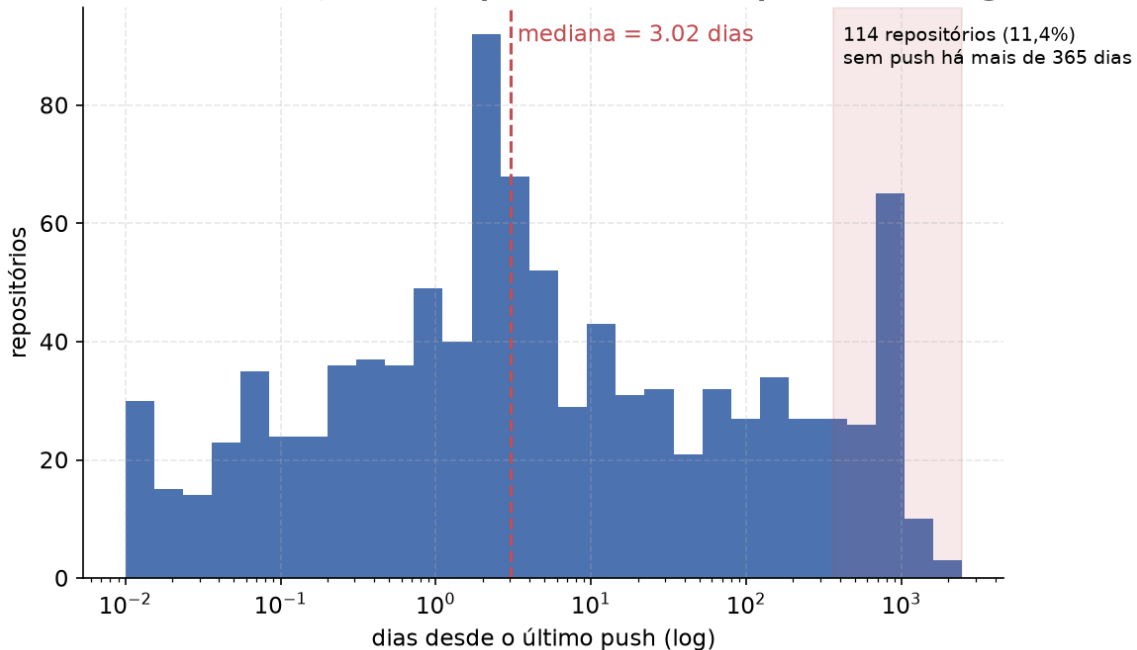
Mediana de 3,02 dias desde o último push (Q1 0,45, Q3 52,1); 60,6% tiveram push na última semana.

114 repositórios (11,4%) estão parados há mais de um ano — cauda de projetos arquivados (atom/atom, adobe/brackets) que seguem estrelados por reputação acumulada.

RQ04 - Repositórios por tempo desde o último push (acumulado)



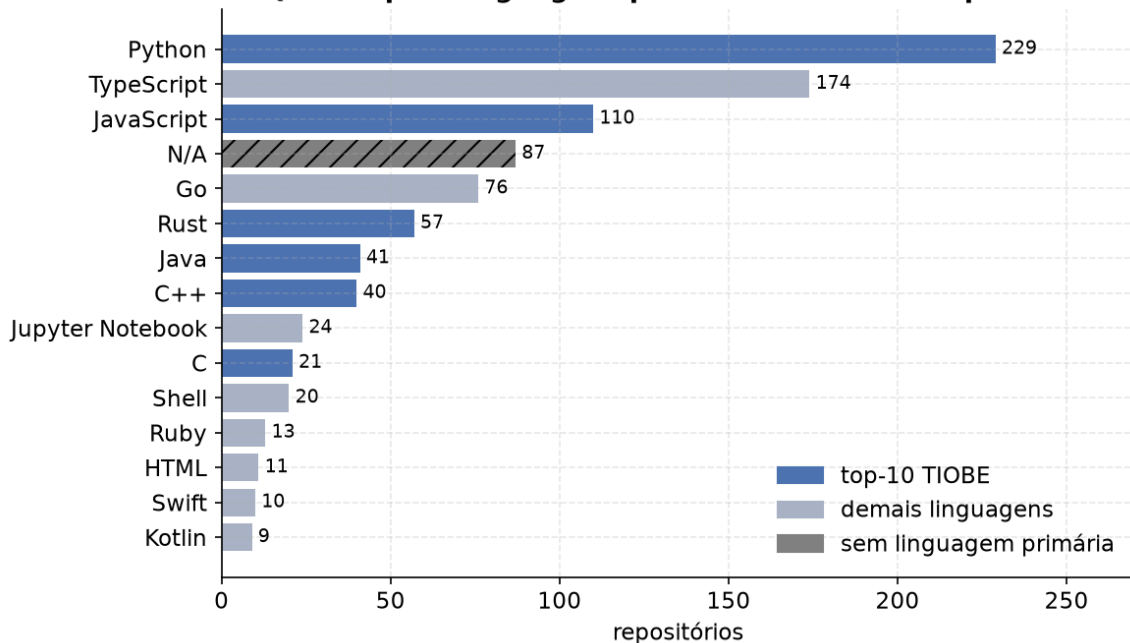
RQ04 - Distribuição do tempo desde o último push (escala log)



RQ05 — Sistemas populares são escritos nas linguagens mais populares?

Python lidera com 229 repositórios (22,9%), seguido de TypeScript (174, 17,4%) e Rust (57, 5,7%). 506 repositórios (50,6%) usam alguma linguagem do top 10 do TIOBE Index; 87 (8,7%) não têm linguagem primária preenchida.

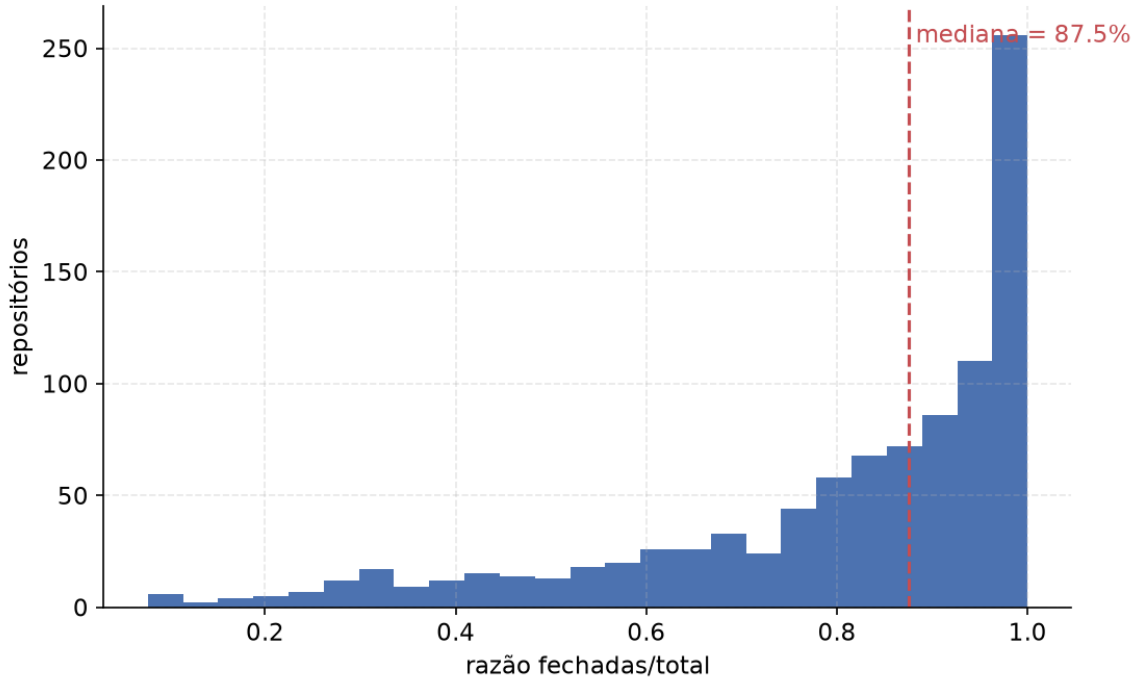
RQ05 - Top 15 linguagens primárias nos 1.000 repositórios



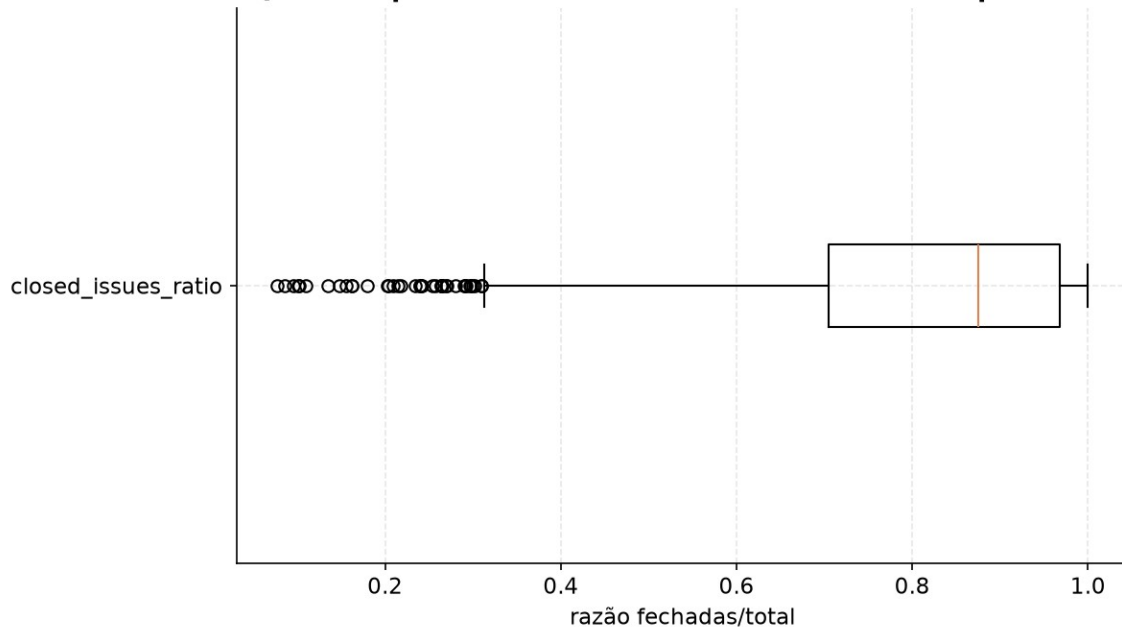
RQ06 — Sistemas populares possuem um alto percentual de issues fechadas?

Com os 957 repositórios válidos (excluídos 43 sem uso do rastreador de issues), a mediana de fechamento é 87,5% (Q1 70,49%, Q3 96,77%); 430 repositórios (44,9%) estão acima de 90%.

RQ06 - Razão de issues fechadas nos 957 repositórios com issues



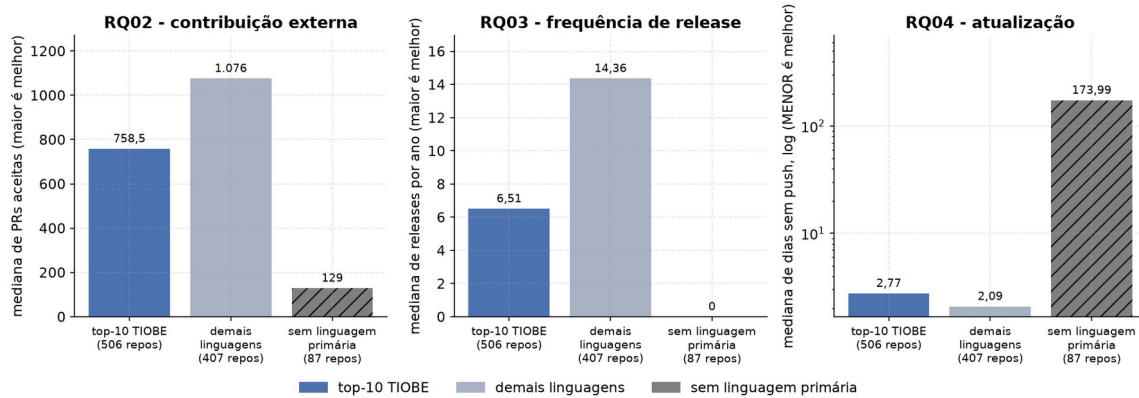
RQ06 - Boxplot da razão de issues fechadas (957 repositórios)



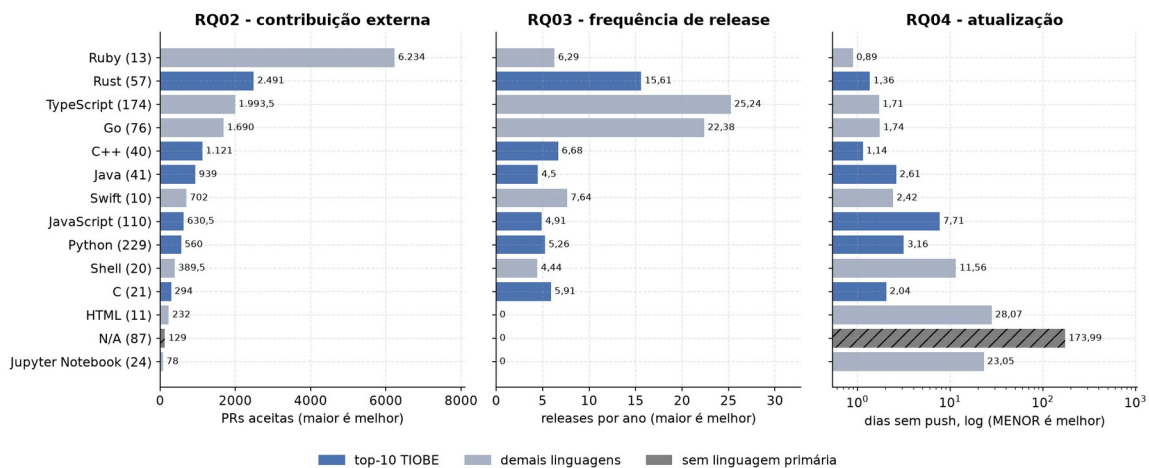
RQ07 — Linguagens mais populares recebem mais contribuição, mais releases e mais atualização?

Resposta negativa nos 1.000: o grupo "demais linguagens" ganha nas três métricas — 1.076 contra 758,5 PRs aceitas, 14,36 contra 6,51 releases por ano e 2,09 contra 2,77 dias sem push (menor é melhor). A idade mediana dos dois grupos é praticamente igual (7,55 x 7,69 anos), então a diferença não é efeito de idade; a causa é a composição do grupo "demais" (TypeScript e Go somam 250 dos 407 repositórios do grupo, e nenhuma das duas está no top-10 TIOBE).

RQ07 - Linguagens mais populares (top-10 TIOBE) vs. demais, nos 1.000 repositórios



RQ07 - Medianas por linguagem primária (linguagens com ao menos 10 repositórios)



4.3 Discussão

RQ01 — Idade dos repositórios

Hipótese: repositórios populares são majoritariamente maduros, mas com uma cauda de projetos recentes que viralizaram rápido.

Resultado: mediana de 7,72 anos (Q1 3,51, Q3 11,34); 345 repositórios (34,5%) com dez anos ou mais e 81 (8,1%) com menos de um ano.

Veredito: confirmada. A mediana quase não se moveu entre as amostras de 100 e 1.000 (8,3 → 7,7 anos) porque a cauda jovem (projetos virais recentes) compensa a cauda madura trazida pelos 1.000 — duas populações convivendo na mesma lista, não uma população homogênea de projetos antigos.

RQ02 — Pull requests aceitas

Hipótese: volume alto de PRs aceitas na mediana, com viés conhecido de projetos que não usam PR do GitHub como fluxo principal aparecerem com zero.

Resultado: mediana de 765,5 PRs aceitas (Q1 175, Q3 3.390); 20 repositórios (2%) com zero.

Veredito: confirmada. O viés do zero se confirma nos casos mais extremos: torvalds/linux e FFmpeg/FFmpeg, dois dos projetos mais populares da amostra, integram contribuição por e-mail/patch fora do fluxo de PR — o zero não significa baixa contribuição externa.

RQ03 — Releases

Hipótese: cadência alta de releases, mas métrica bimodal: um grupo grande não usa GitHub Releases (por não ser software ou por versionar por tag); entre os que publicam, a cadência é alta.

Resultado: 275 repositórios (27,5%) sem release; entre os 725 que publicam, mediana de 14,94 releases/ano (6,78 considerando todos).

Veredito: confirmada, com recorte reinterpretado. A bimodalidade se sustenta nos dois extremos. A queda de 40% (amostra de 100) para 27,5% (1.000) não é inconsistência de coleta: quanto maior o corte de popularidade, maior a concentração de repositório de conteúdo (listas, livros, roadmaps), que não publica release por não ser software — a cauda dos 1.000 traz proporcionalmente mais biblioteca e ferramenta, que publica.

RQ04 — Atualização

Hipótese: push frequente (mediana de poucos dias) convivendo com uma cauda de projetos arquivados que seguem estrelados por reputação acumulada.

Resultado: mediana de 3,02 dias desde o último push; 60,6% com push na última semana; 11,4% parados há mais de um ano.

Veredito: confirmada. As duas partes da hipótese aparecem nos 1.000. A escolha de pushedAt em vez de updatedAt é decisiva: por updatedAt (que muda com qualquer star, label ou edição de descrição) a mediana cairia para 43 minutos e a cauda de abandonados desapareceria por completo do gráfico.

RQ05 — Linguagens populares

Hipótese: os projetos de maior sucesso são desenvolvidos predominantemente nas linguagens que dominam o mercado (referência TIOBE Index), pelo maior ecossistema de bibliotecas e de contribuidores.

Resultado: 506 repositórios (50,6%) usam linguagem do top 10 do TIOBE; Python lidera (229), seguido de TypeScript (174) e JavaScript (110). Java (41) e C++ (40), gigantes de mercado, ficam abaixo de Go (76) e Rust (57).

Veredito: parcialmente confirmada. A concentração existe, mas não nas linguagens que o TIOBE sugere: a amostra é dominada pelo ecossistema de dados e web, enquanto Java e C++ ficam abaixo de linguagens emergentes na plataforma GitHub.

RQ06 — Issues fechadas

Hipótese: taxa de issues fechadas muito alta (mediana acima de 80%), refletindo manutenção ativa.

Resultado: excluídos os 43 repositórios com total_issues == 0, restaram 957; mediana de 87,5% (Q1 70,49%, Q3 96,77%), com 430 (44,9%) acima de 90%.

Veredito: confirmada. A mediana supera os 80% da hipótese mesmo sem os 43 casos artificiais que inflariam a taxa para 0%.

RQ07 — Contribuição, releases e atualização por linguagem

Hipótese: a leitura natural é que sim — linguagem popular teria maior ecossistema e mais contribuidores, puxando as três métricas para cima (mesmo argumento estrutural da RQ05).

Resultado: nos 1.000, o grupo "demais linguagens" ganha nas três métricas ao mesmo tempo (1.076 x 758,5 PRs aceitas; 14,36 x 6,51 releases/ano; 2,09 x 2,77 dias sem push). A idade mediana dos dois grupos é praticamente igual (7,55 x 7,69 anos), então não é efeito de idade.

Veredito: refutada. A causa não é a popularidade da linguagem em si, mas a composição do grupo "demais": TypeScript (174) e Go (76) somam 250 dos 407 repositórios do grupo, e tirando as duas o grupo cai abaixo do top-10 TIOBE nas três métricas. O corte "top-10 TIOBE vs. demais" acaba separando linguagem de engenharia de software moderna do resto, não linguagem popular de impopular — limitação da fonte adotada, já que o TIOBE mede uso na indústria (incluindo código fechado) e o recorte aqui é de repositórios open-source.

RQ extra — A idade explica o número de estrelas?

Hipótese: se sistemas populares tendem a ser antigos (RQ01), os mais antigos deveriam ser também os mais estrelados.

Resultado: correlação praticamente nula entre idade e estrelas (Pearson 0,112; Spearman 0,003, nos 100 repositórios). A faixa com menos de 1 ano de vida tem a maior mediana de estrelas de toda a amostra (195.075, contra 172.660 nos repositórios com mais de 10 anos).

Veredito: refutada. Idade não é o que produz estrelas: convivem duas populações na mesma lista — projetos maduros que acumularam estrelas ao longo de uma década e projetos virais recentes (majoritariamente ligados a IA e agentes) que chegaram lá em meses. Esse resultado reforça a leitura da RQ01: a mediana de idade é alta, mas a métrica sozinha esconde a cauda jovem que também é popular.

Ameaças à validade: a amostra é um recorte de popularidade (estrelas no GitHub), não uma amostra aleatória do universo de software — os resultados não generalizam para projetos fechados ou pouco estrelados. Métricas agregadas por totalCount (releases, PRs, issues) não distinguem fluxos alternativos de contribuição (patch por e-mail, versionamento por tag), tratados aqui como recortes declarados em vez de dado ausente. A comparação por linguagem (RQ07) depende do corte de popularidade adotado para "linguagem mais popular" (TIOBE), que mede uso de mercado e não composição do open-source — outra fonte (GitHub Octoverse, por exemplo) provavelmente mudaria a composição dos grupos comparados.

A inovação da seção 3.6 (RQ extra sobre idade x estrelas) aprofunda a RQ01 em vez de contradizê-la: ela mostra que a mediana de idade alta não implica que idade "produz" popularidade, revelando a segunda população (viral/recente) que a métrica de RQ01 sozinha não deixava visível.

5. Conclusão

Das sete questões do enunciado, seis hipóteses informais se confirmaram (RQ01, RQ02, RQ03, RQ04, RQ06 e, parcialmente, RQ05) e uma foi refutada pelos dados (RQ07): linguagem popular no mercado (TIOBE) não se traduz em mais contribuição, mais releases ou mais atualização nos repositórios open-source mais estrelados — o corte que realmente separa os grupos é composição (TypeScript e Go concentram o grupo "demais"), não popularidade de mercado. A RQ extra proposta pelo grupo reforça essa mesma lição em outro eixo: idade explica a mediana alta da RQ01, mas não o número de estrelas — a amostra convive com duas populações, maduros e virais.

Escala mudou o tipo de resposta, não o veredito: passar de 100 para 1.000 repositórios não inverteu nenhuma hipótese, mas revelou fenômenos que a amostra pequena escondia — os gigantes que não usam o fluxo de PR do GitHub (RQ02), a fração estável de projetos sem release (RQ03), a cauda de arquivados que só pushedAt revela (RQ04) e o grupo "sem linguagem" grande o bastante para virar recorte próprio (RQ05/RQ07). Isso reforça que popularidade não é proxy de saúde uniforme de projeto.

Limitações do estudo: a amostra é um recorte de popularidade por estrela, não uma amostra aleatória do universo de software, e não generaliza para código fechado. A fonte TIOBE mede uso de mercado, enquanto a amostra é de open-source — as duas populações divergem, e isso pesa especialmente na leitura da RQ07. O valor de RQ06 registrado na validação da S02 (89,28%) diverge do recalculado sobre o snapshot consolidado usado neste relatório (87,5%) porque as duas medições foram feitas em momentos diferentes sobre dados vivos e fora do controle do grupo: `issues.totalCount` e `issues(states: CLOSED).totalCount` mudam a cada issue aberta ou fechada nos 1.000 repositórios entre uma coleta e outra — o mesmo efeito já documentado para releases (item 6 das limitações do lab01/README.md, ex.: `vercel/next.js` 3799 → 3800 releases entre duas execuções). Não é erro de coleta nem inconsistência de cálculo; é a natureza de medir um sistema em produção, e este relatório usa sempre o valor reproduzível a partir do CSV consolidado atual.

Com mais tempo, o grupo aprofundaria duas frentes: (i) repetir a coleta em uma data posterior para tratar a RQ07 como série temporal, já que a explicação por composição de linguagem sugerida aqui é uma fotografia única; e (ii) investigar se a fonte de "linguagem mais popular" (TIOBE vs. GitHub Octoverse) muda o resultado da RQ07 o suficiente para valer a pena reportar as duas em paralelo. Um ajuste técnico menor, sem impacto nos números do relatório, também ficou pendente: a borda final dos histogramas em escala log (RQ02/RQ03) corta o repositório de maior valor por arredondamento de ponto flutuante.

5. Referências

- ZUSE, Horst. A framework of software measurement. Walter de Gruyter, 2013.
- TIOBE Software. TIOBE Index. Disponível em: <https://www.tiobe.com/tiobe-index/>. Acesso em: 2026.
- GITHUB, Inc. GitHub GraphQL API documentation. Disponível em: <https://docs.github.com/en/graphql>. Acesso em: 2026.

- GITHUB, Inc. GitHub Projects documentation. Disponível em:
<https://docs.github.com/en/issues/planning-and-tracking-with-projects>. Acesso em: 2026.
-